

ProbAct: A Probabilistic Activation Function for Deep Neural Networks

Kumar Shridhar ♣, Joonho Lee ♣, Hideaki Hayashi ♣, Purvanshi Mehta •
 , Brian Kenji Iwana ♣, Seokjun Kang ♣, Seiichi Uchida ♣, Sheraz Ahmed ◊
 , Andreas Dengel ◊

♣ University of Kaiserslautern, Germany ♣ Kyushu University, Japan
 • University of Rochester, USA ◊ DFKI, Kaiserslautern

1

Abstract

Activation functions play an important role in training artificial neural networks. The majority of currently used activation functions are deterministic in nature, with their fixed input-output relationship. In this work, we propose a novel probabilistic activation function, called *ProbAct*. *ProbAct* is decomposed into a mean and variance and the output value is sampled from the formed distribution, making *ProbAct* a stochastic activation function. The values of mean and variances can be fixed using known functions or trained for each element. In the trainable *ProbAct*, the mean and the variance of the activation distribution is trained within the back-propagation framework alongside other parameters. We show that the stochastic perturbation induced through *ProbAct* acts as a viable generalization technique for feature augmentation. In our experiments, we compare *ProbAct* with well-known activation functions on classification tasks on different modalities: Images (CIFAR-10, CIFAR-100 and STL-10) and Text (Large Movie Review). We show that *ProbAct* increases the classification accuracy by +2-3% compared to ReLU or other conventional activation functions on both original datasets and when datasets are reduced to 50% and 25% of the original size. Finally, we show that *ProbAct* learns an ensemble of models by itself that can be used to estimate the uncertainties associated with the prediction and provides robustness to noisy inputs.

1 Introduction

Activation functions add non-linearity to neural networks making them learn complex functional mappings from data [1]. Different activation functions with different characteristics have been proposed. Sigmoid [2] and hyperbolic tangent (Tanh) were the popular ones during the early usage of neural networks [3] mainly due to their monotonicity, continuity, and bounded properties. In recent times, the Rectified Linear Unit (ReLU) [4] has become an extremely popular activation function for neural networks. Several variants of ReLU have been proposed, e.g., Leaky ReLU [5], Parametric ReLU (PReLU) [6], and Exponential Linear Unit (ELU) [7].

However, all of these are deterministic activation functions with fixed input-output relationships. In this work, we propose a new activation function, called *ProbAct*, which is not only trainable but also

1

♣ k_shridhar16@cs.uni-kl.de

♣ {joonho.lee, hayashi, brian, seokjun.kang, uchida}@human.ait.kyushu-u.ac.jp

• pmehta@ur.rochester.edu ◊ {sheraz.ahmed, andreas.dengel}@dfki.de

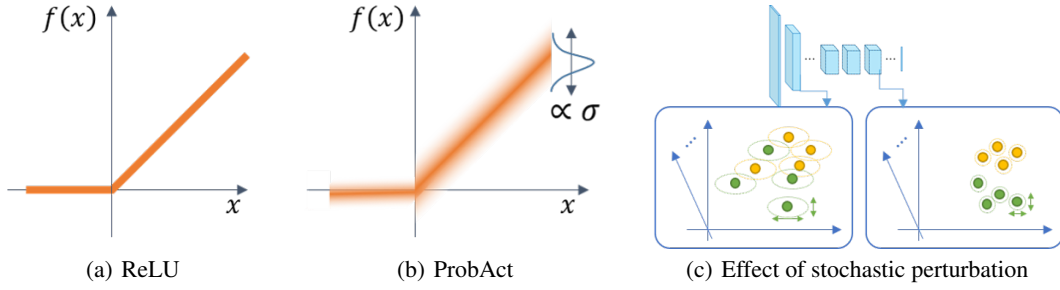


Figure 1: Comparison of (a) ReLU and (b) the proposed activation function. (c) is the effect of stochastic perturbation by ProbAct at feature spaces in a neural network.

stochastic in nature. The idea of ProbAct is inspired by the stochastic behavior of biological neurons. Noise in neuronal spikes can arise due to uncertain bio-mechanical effects [8]. We try to emulate a similar behavior in the information flow to the neurons by injecting stochastic sampling from a Gaussian distribution to the activations. Consequently, even for the same input value x , the output value from ProbAct varies stochastically — a capability conventional activation functions do not offer.

The induced perturbations prove to be effective in avoiding overfitting during training, thus yielding better generalizations. Since the operation is a resemblance to feature augmentation, we call it *augmentation-by-activation*. Furthermore, we show that ProbAct improves the classification accuracy by 2-3% compared to ReLU or other conventional activation functions on established image datasets and 1-2% on text datasets. The main contributions of our work are as follows:

- We introduce a novel activation function, called ProbAct, whose output undergoes stochastic perturbation.
- We propose a novel method of governing the stochastic perturbation with parameters trained through back-propagation.
- We show that ProbAct improves the performance on various visual and textual classification tasks while generalizing well on reduced datasets.
- We also show that the improvement by ProbAct is realized by the augmentation-by-activation, which acts as a stochastic regularizer to prevent overfitting of the network and acts as a feature augmentation method.
- Finally, we demonstrate that ProbAct learns an ensemble of models by itself, allowing the estimation of predictive uncertainties and robustness to noisy data.

2 Related Work

2.1 Activation Functions

Various approaches have been applied in the past to create a desired activation function. Research on activation functions can be broadly separated into two approaches: fixed activation functions, and adaptive activation functions.

Fixed activation functions are constant pre-determined functions such as sigmoid, tanh, and ReLU. In particular, ReLU has led to significant improvements in neural network performance. However, ReLU faces a dead neuron problem and variants of ReLU were suggested to solve this problem. For example, Leaky ReLUs [5] use a fixed $0.01x$ value for $x < 0$. Other activation functions like Swish [9] and Exponential Linear Sigmoid SquasHing (ELiSH) [10] take a different approach and use bounded negative regions.

Adaptive activation functions use trainable parameters in order to optimize the activation function. For example, Parametric ReLUs (PReLU) [6] are similar to Leaky ReLUs but with a trainable parameter instead of a fixed value. In addition, S-shape ReLU (SReLU) [11] and Parametric ELU (PELU) [12] were suggested to improve the performance of conventional ReLU functions.

2.2 Generalization and Stochastic Methods

There has been less research on stochastic activation functions due to expensive sampling processes. Noisy activation functions [13] tried to deal with these problems by adding noises to the non-linearity in proportion to the magnitude of saturation of the non-linearity. RReLU [5] uses Leaky ReLUs with randomized slopes during training and a fixed slope during testing.

There are many other generalization techniques that use random distributions or stochastic functions. For example, dropout [14] generalizes by removing connections at random during training and can be interpreted as a way of model averaging. [15] suggested Random Self-Ensemble (RSE) by combining randomness and ensemble learning, and [16] proposed back-propagating noise to the hidden layers. Furthermore, the effects of adding noise to inputs [17, 18], the gradient [19, 20], and weights [18, 21–23] have been studied. However, we adopt the concept that stochastic neurons with sparse representations allow internal regularization as shown by [24].

Neural networks using point estimates as weights and fixed activations always result in the same prediction with no uncertainty estimates. Introducing Bayesian inference on weights of a network [25–28] is one way to estimate the predictive uncertainty. [29–31] show other approaches to estimate the predictive uncertainty and decompose it according to its origin. However, Bayesian neural networks increase the number of parameters as weights are represented by means of a parametric model. Deep ensembles [32] uses non-Bayesian networks to achieve similar results but training N networks for any large task is computationally very expensive. We show that ProbAct acts as ensembles of networks that can be used to estimate predictive uncertainties with only a few additional parameters.

[33] proposed natural parameter networks (NPN), where exponential-family distributions represented the inputs, targets and weights. [34] and [35] also treat the output of activation function as distributions rather than just deterministic values. However, ProbAct’s variance is input-independent while [33–35] produce input-dependent variance, making ProbAct faster and less computationally expensive.

3 ProbAct: A Stochastic Activation Function

Every layer of a neural network computes its output y for the given input $\mathbf{x}$:

$$y = f(\mathbf{w}^T \mathbf{x}), \quad (1)$$

where $\mathbf{w}$ is the weight vector of the layer and $f(\cdot)$ can be any activation function, such as ProbAct. ProbAct is defined as:

$$f(\mathbf{x}) = \mu(\mathbf{x}) + z, \quad (2)$$

where $\mu(\mathbf{x})$ is a static or learnable mean (for example, $\mu(\mathbf{x}) = \max(0, \mathbf{x})$ if it is static ReLU) and the perturbation term z is:

$$z = \sigma \epsilon. \quad (3)$$

The perturbation parameter σ is a fixed or trainable value which specifies the range of stochastic perturbation and ϵ is a random value sampled from a normal distribution $\mathcal{N}(0, 1)$. The value of σ is either determined manually or trained along with other network parameters (i.e., weights) with simple implementation. With decreasing σ , ProbAct converges to its mean function $\mu(\mathbf{x})$. If $\sigma \rightarrow 0$, ProbAct behaves the same as its mean function. Example: if the mean is a fixed ReLU function, then ProbAct acts a generalization of ReLU in that case.

3.1 Setting the Parameter for Mean

The mean function $\mu(\mathbf{x})$ is trained for every input $\mathbf{x}$, i.e. element-wise. However, learning the mean value with zero or random initialization takes unnecessarily long to converge. So, we propose ways for mean initialization.

3.1.1 Mean initialization:

We propose initialization of μ with known functions such as ReLU with $\mu(\mathbf{x}) = \max(0, \mathbf{x})$. Besides ReLU, any known functions can be used as an initializer. We use ReLU for its simplicity and good convergence behavior.